

Bridge Design Pattern

Study notes - structural pattern, first-principles walkthrough with FAANG interview angle

1. The Problem (First Principles)

Real-life analogy: a universal remote control should work with a TV, a Radio, or a Sound System, and should support both a Basic remote and an Advanced remote (with mute). If you handle this purely through inheritance, you are tempted to write RemoteControlForTV, RemoteControlForRadio, AdvancedRemoteControlForTV, AdvancedRemoteControlForRadio... Two remote features x two devices = 4 classes. Add a SoundSystem: 6 classes. Add a VoiceRemoteControl feature: 9 classes.

This is **class explosion** - two independent dimensions of variation (remote type, device type) multiplied together by inheritance instead of kept separate. A real universal remote does not work this way: the remote (front end) is built completely separately from the device's electronics. The remote just sends signals like "power" or "volume up", and any compatible device can receive them. A **bridge** connects the two hierarchies without merging them.

The problem Bridge solves: **two hierarchies need to vary independently, but naive inheritance multiplies them together.**

2. Standard (GoF) Definition

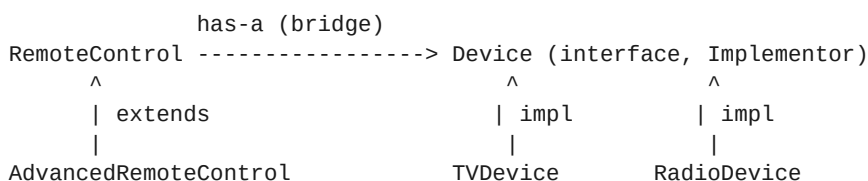
"Decouple an abstraction from its implementation so that the two can vary independently."

It is a **structural** pattern. Instead of the abstraction inheriting its behavior, it holds a reference to an implementor object and delegates to it - composition over inheritance, applied proactively at design time to prevent an explosion, rather than reactively (as in Adapter) to fix an incompatibility.

3. Structure & Participants

Four roles, three relationship types - the richest structure of the three patterns covered so far, combining inheritance and a has-a bridge:

- **Abstraction** (RemoteControl) - defines high-level operations. Holds a reference to a Device and delegates to it; does not implement device-specific logic itself.
- **Refined Abstraction** (AdvancedRemoteControl) - extends RemoteControl with extra features (e.g. mute()). Grows only the remote hierarchy.
- **Implementor** (Device) - the interface defining low-level operations (turnOn(), setVolume()) any device must support.
- **Concrete Implementors** (TVDevice, RadioDevice) - actual device-specific implementations. New devices only add to this hierarchy.



The bridge is the single has-a arrow. It lets both hierarchies expand independently - a new device or new remote feature adds ONE class, not

a whole cross-product of classes.

Text rendering of the UML structure: RemoteControl -> Device (association/composition, the bridge), AdvancedRemoteControl -> RemoteControl (inheritance), TVDevice/RadioDevice -> Device (realization).

4. Code Example

The problem - without Bridge (inheritance-only, class explosion)

```
// Trying to handle "feature level" and "device type" through pure inheritance
class RemoteControlForTV {
    void turnOn() { /* TV-specific power-on logic */ }
    void turnOff() { /* ... */ }
}

class RemoteControlForRadio {
    void turnOn() { /* Radio-specific power-on logic */ }
    void turnOff() { /* ... */ }
}

class AdvancedRemoteControlForTV extends RemoteControlForTV {
    void mute() { /* ... */ }
}

class AdvancedRemoteControlForRadio extends RemoteControlForRadio {
    void mute() { /* ... */ }
}

// Add a SoundSystem, and you need 2 more classes.
// Add a VoiceRemoteControl feature, and you need 3 more classes (one per device).
```

Two dimensions of variation (remote type x device type) are tangled into a single inheritance tree. Every new item in either dimension multiplies your class count.

The solution - with Bridge

```
// Implementor hierarchy - the "device" side, completely independent
interface Device {
    void turnOn();
    void turnOff();
    void setVolume(int percent);
}

class TVDevice implements Device {
    public void turnOn() { System.out.println("TV powering on"); }
    public void turnOff() { System.out.println("TV powering off"); }
    public void setVolume(int percent) { System.out.println("TV volume: " + percent); }
}

class RadioDevice implements Device {
    public void turnOn() { System.out.println("Radio powering on"); }
    public void turnOff() { System.out.println("Radio powering off"); }
    public void setVolume(int percent) { System.out.println("Radio volume: " + percent); }
}

// Abstraction hierarchy - the "remote" side, completely independent
class RemoteControl {
    protected Device device; // <-- the bridge: composition, not inheritance
    public RemoteControl(Device device) { this.device = device; }
```

```

    public void togglePower() { device.turnOn(); }
    public void volumeUp()    { device.setVolume(75); }
}

class AdvancedRemoteControl extends RemoteControl {
    public AdvancedRemoteControl(Device device) { super(device); }
    public void mute() { device.setVolume(0); } // new remote feature, works with ANY device
}

// Usage - mix and match freely, no class explosion
RemoteControl basicForTV = new RemoteControl(new TVDevice());
basicForTV.togglePower();

AdvancedRemoteControl advancedForRadio = new AdvancedRemoteControl(new RadioDevice());
advancedForRadio.mute();

```

AdvancedRemoteControl works with TVDevice, RadioDevice, or any future device because it only ever talks to the Device interface. Add a SoundSystemDevice: zero changes to the remote hierarchy. Add a VoiceRemoteControl: zero changes to the device hierarchy.

5. Variants / Related Forms

Variant 1 - Static Bridge (implementor fixed at construction)

The classic form above: the Device is passed into the RemoteControl constructor once and never changes for that remote's lifetime.

Analogy: a remote factory-paired to a specific TV. It always talks to that one device.

Variant 2 - Dynamic Bridge (implementor swappable at runtime)

Add a setter (setDevice(Device newDevice)) so the same abstraction object can be re-pointed at a different implementor while the program is running.

Analogy: a genuinely universal remote where pressing a "pairing" button re-syncs it to a completely different TV or soundbar, without buying a new remote.

```

class RemoteControl {
    protected Device device;
    public void setDevice(Device device) { this.device = device; } // re-bridge at runtime
}

```

Variant 3 - Bridge as a Generalization of Strategy

Structurally, Bridge and Strategy look almost identical - an object holds a reference to an interchangeable interface and delegates to it. Strategy (behavioral) typically has one concrete class on the context side and swaps just the algorithm. Bridge (structural) deliberately maintains inheritance hierarchies on both sides.

Analogy: Strategy is choosing a specific route for a single, fixed car (a GPS trying driving vs walking vs transit). Bridge is designing the entire concept of "a vehicle" and "a routing engine" as two separate product lines from day one, so any vehicle pairs with any routing engine.

Variant	Implementor binding	Real-life analogy
---------	---------------------	-------------------

Static Bridge	Fixed at construction	Remote factory-paired to one TV
Dynamic Bridge	Swappable at runtime	Universal remote re-synced to a new device
Bridge-as-Strategy	Same shape, narrower intent	GPS route choice vs whole vehicle + engine product lines

6. Real-World Use Cases

- **JDBC:** Connection/Statement (abstraction) bridges to vendor-specific driver implementations (Oracle, MySQL, PostgreSQL) - your JDBC code never changes when you switch databases.
- **Java AWT peer architecture:** Button, Checkbox (abstraction widgets) bridge to platform-specific ButtonPeer, CheckboxPeer implementations on Windows, macOS, Linux.
- **Cross-platform GUI/mobile frameworks:** a shared UI abstraction bridging to native rendering implementors per platform (iOS vs Android).
- **Notification systems:** a Notification abstraction (Alert, Reminder) bridging to multiple delivery channels (Email, SMS, Push) without a class per combination.
- **Shape rendering** (the classic GoF example): Shape (Circle, Square) bridging to DrawingAPI (Vector renderer, Raster renderer).
- **Payment abstraction across providers:** a PaymentMethod abstraction bridging to PaymentGateway implementors (Stripe, PayPal, Razorpay) when both payment kind and provider vary independently.

7. FAANG Interview Angle - Bridge vs Its Cousins

- **vs Adapter:** Adapter is applied after the fact - retrofitting compatibility between an existing client and an existing incompatible class, usually because you do not control one side. Bridge is designed up front - you deliberately split two hierarchies before any incompatibility problem exists, specifically to prevent combinatorial class explosion as both sides grow.
- **vs Strategy:** nearly identical structure, different scope of intent. Strategy swaps a single algorithm on one side of a single class. Bridge maintains two parallel inheritance hierarchies that both need room to grow independently.
- **vs Abstract Factory:** often used together, not as alternatives. Abstract Factory commonly constructs the correct ConcreteImplementor to hand to the Abstraction at runtime (e.g. choosing WindowsButtonPeer vs MacButtonPeer based on OS) - Abstract Factory picks the implementor, Bridge is the structure that lets the abstraction use whichever one it is given.
- **Common curveball:** "Isn't Bridge just composition over inheritance?" Good answer: yes, that is literally the mechanism - but the specific intent is preventing a combinatorial class explosion across two independently varying dimensions, narrower than "avoid inheritance in general."
- **Another common question:** "When would you introduce Bridge versus just live with a few extra subclasses?" Good answer: Bridge earns its complexity when there are (or will be) genuinely two or more independent dimensions of variation that keep growing; for a fixed, small combination count plain inheritance may be simpler and Bridge would be over-engineering.

8. Memory Map (Quick Recall)

```

|
+-- PROBLEM: Two independent dimensions of variation, handled with inheritance,
|   multiply into a combinatorial class explosion
|   (analogy: universal remote hardware vs devices - welded together means
|   every new remote feature x every new device = a whole new class)
|
+-- DEFINITION: Decouple an abstraction from its implementation so that the two
|   can vary independently
|
+-- 4 PARTICIPANTS
|   +-- Abstraction      -> RemoteControl, has-a Implementor reference
|   +-- Refined Abstraction -> AdvancedRemoteControl, extends Abstraction
|   +-- Implementor      -> Device interface, low-level operations
|   +-- Concrete Implementor -> TVDevice, RadioDevice, implement Implementor
|
+-- 3 VARIANTS
|   +-- Static Bridge    -> implementor fixed at construction
|   +-- Dynamic Bridge   -> implementor swappable at runtime (setDevice)
|   +-- Bridge-as-Strategy -> same shape, narrower intent (one algorithm swap)
|
+-- REAL EXAMPLES: JDBC drivers, AWT peers, cross-platform GUIs, notification
|   channels, shape/rendering-API (classic GoF example), multi-provider payments
|
+-- INTERVIEW DIFFERENTIATORS
|   +-- vs Adapter      -> designed up front, not retrofitted after the fact
|   +-- vs Strategy     -> both sides have subclass hierarchies, not just one
|   +-- vs Abstract Factory -> often paired: factory picks which implementor to inject
|   +-- is composition over inheritance -> but specifically to stop class explosion

```

One sentence to remember: Bridge splits two things that grow independently into two hierarchies connected by a single has-a line, so growth in one never multiplies into the other.